

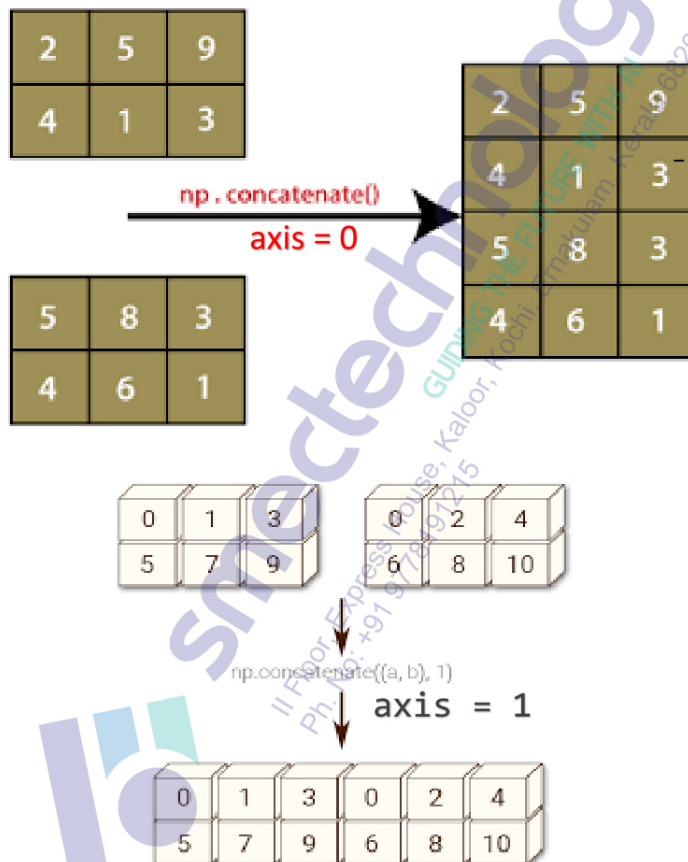
Joining Arrays

1. `numpy.concatenate`

Concatenation refers to joining. This function is used to join two or more arrays of the same shape along a specified axis. The function takes the following parameters.

Syntax:

```
numpy.concatenate((a1, a2, ...), axis)
```



```
import numpy as np  
a = np.array([[1,2],[3,4]])
```

```
print('First array:')
print(a)

b = np.array([[5,6],[7,8]])
print('Second array:')
print(b)

# both the arrays are of same dimensions
print('Joining the two arrays along axis 0:')
print(np.concatenate((a, b)))
print('Joining the two arrays along axis 1:')
print(np.concatenate((a, b), axis=1))
```

output:

First array:

```
[[1 2]
 [3 4]]
```

Second array:

```
[[5 6]
 [7 8]]
```

Joining the two arrays along axis 0:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Joining the two arrays along axis 1:

```
[[1 2 5 6]]
```

```
[3 4 7 8]]
```

2. numpy.hstack

Variants of numpy.stack function to stack so as to make a single array horizontally.

```
import numpy as np
a = np.array([[1,2],[3,4]])
print('First array:')
print(a)
```

```
b = np.array([[5,6],[7,8]])
print('Second array:')
print(b)
```

```
print('Horizontal stacking:')
c = np.hstack((a,b))
print(c)
```

output:

First array:

```
[[1 2]
 [3 4]]
```

Second array:

```
[[5 6]
 [7 8]]
```

Horizontal stacking:

```
[[1 2 5 6]
 [3 4 7 8]]
```

3. numpy.vstack

Variants of numpy.stack function to stack so as to make a single array vertically.

```
import numpy as np
a = np.array([[1,2],[3,4]])
print('First array:')
print(a)
b = np.array([[5,6],[7,8]])
print('Second array:')
print(b)
print('Vertical stacking:')
c = np.vstack((a,b))
print(c)
```

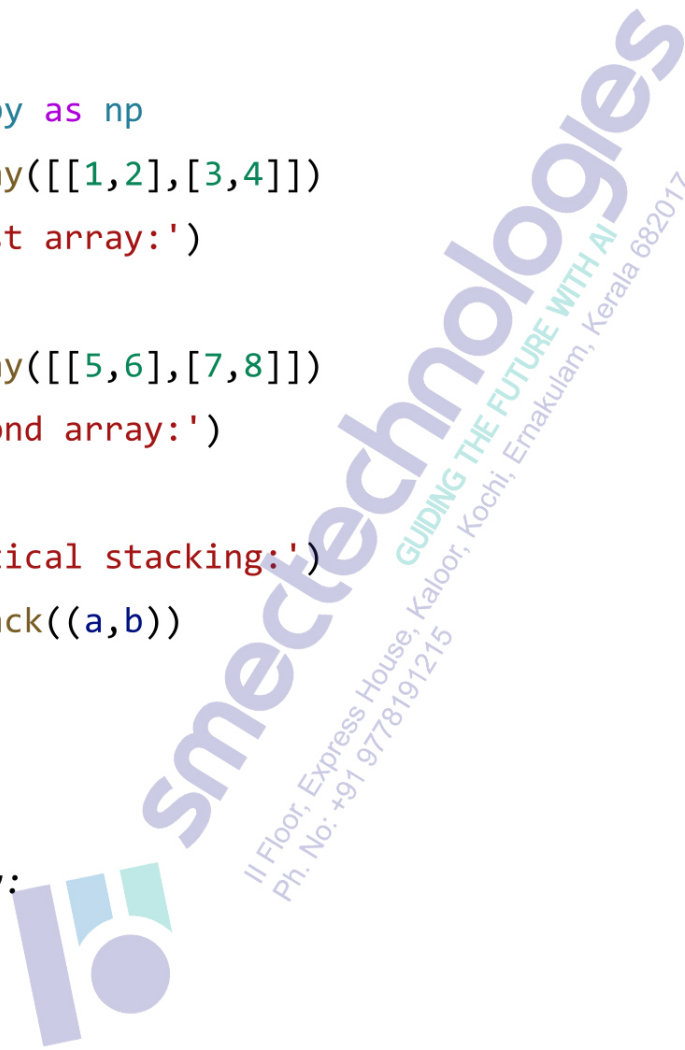
output:

First array:

```
[[1 2]
 [3 4]]
```

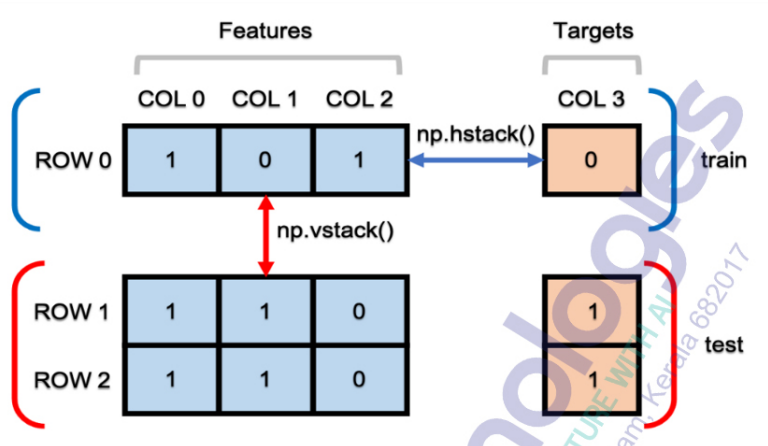
Second array:

```
[[5 6]
 [7 8]]
```



Vertical stacking:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```



We will learn about features targets in machine learning section

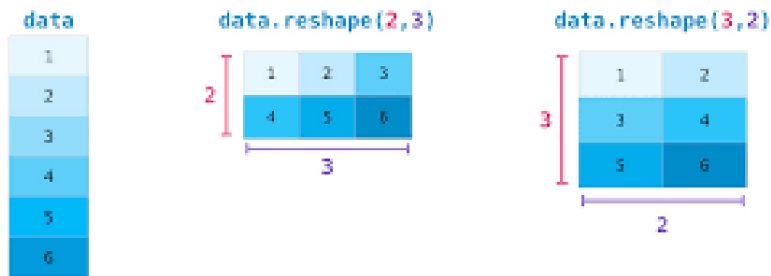
Array Manipulation

1. numpy.reshape

This function gives a new shape to an array without changing the data. It accepts the following parameters –

Syntax:

```
numpy.reshape(arr, newshape, order)
```



```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print(a)
b = a.reshape(3,2)
print(b)
```

output:

```
[[1 2 3]
 [4 5 6]]
```

```
[[1 2]
 [3 4]
 [5 6]]
```

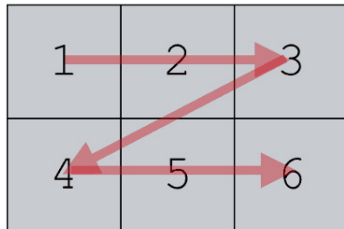
2. numpy.ndarray.flatten

This function returns a copy of an array collapsed into one dimension. The function takes the following parameters.

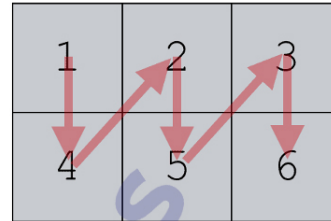
Syntax:

```
ndarray.flatten(order)
```

'C' flattens in row-first order (C-style)



'F' flattens in column-first order (Fortran-style)



```
import numpy as np
a = np.arange(8).reshape(2,4)
print('The original array is:')
print(a)

# default is column-major
print('The flattened array is:')
print(a.flatten())
print('The flattened array in F-style ordering:')
print(a.flatten(order='F'))
```

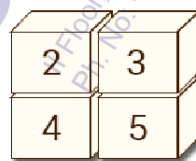
output:

The original

```
[[0 1 2 3]
 [4 5 6 7]]
```

The

```
[0 1 2 3 4 5
```



flatten()



array is:

flattened array is:

```
6 7]
```

The flattened array in F-style ordering:

```
[0 4 1 5 2 6 3 7]
```

3. `numpy.ndarray.flat`

This function returns a 1-D iterator over the array. It behaves similar to Python's built-in iterator.

```
import numpy as np
a = np.arange(8).reshape(2,4)
print('The original array:')
print(a)
print('After applying the flat function:')
# returns element corresponding to index in flattened array
print(a.flat[5])
```

output:

The original array:

```
[[0 1 2 3]
```

```
 [4 5 6 7]]
```

After applying the flat function:

5

4. `Splitting Arrays`

This function divides the array into subarrays along a specified axis.

The function takes three parameters.

Syntax:

```
numpy.split(arr, indices_or_sections, axis)
```

- `arr` : Input array to be split
- `indices_or_sections` : Can be an integer, indicating the number of equal sized subarrays to be created from the input array. If this parameter is a 1-D array, the entries indicate the points at which a new subarray is to be created.
- `axis` : Default is 0

```
import numpy as np
```

```
a = np.arange(9)
```

```
print('First array:')
```

```
print(a)
```

```
print('Split the array in 3 equal-sized subarrays:')
```

```
b = np.split(a,3)
```

```
print(b)
```

```
print('Split the array at positions indicated in 1-D array:')
```

```
b = np.split(a,[4,7])
```

```
print(b)
```

output:

First array:

```
[0 1 2 3 4 5 6 7 8]
```

Split the array in 3 equal-sized subarrays:

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

Split the array at positions indicated in 1-D array:

[array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8])]

5. numpy.resize

This function returns a new array with the specified size. If the new size is greater than the original, the repeated copies of entries in the original are contained. The function takes the following parameters.

Syntax:

```
numpy.resize(arr, shape)
```

- *arr* : Input array to be resized
- *shape* : New shape of the resulting array

```
import numpy as np
arr = np.array([[1,2,3],[4,5,6]])
print('First array:')
print(arr)

print('The shape of first array:')
print(arr.shape)

arr1 = np.resize(arr, (3,2))
print('Second array:')
print(arr1)
```

```
print('The shape of second array:')  
print(arr1.shape)
```

Observe that first row of a is repeated in b since size is bigger

```
print('Resize the second array:')  
arr1 = np.resize(arr,(3,3))  
print(arr1)
```

output:

First array:

```
[[1 2 3]  
 [4 5 6]]
```

The shape of first array:

```
(2, 3)
```

Second array:

```
[[1 2]  
 [3 4]  
 [5 6]]
```

The shape of second array:

```
(3, 2)
```

Resize the second array:

```
[[1 2 3]  
 [4 5 6]  
 [1 2 3]]
```

6. numpy.append

This function adds values at the end of an input array. The append operation is not inplace, a new array is allocated. Also the dimensions of the input arrays must match otherwise ValueError will be generated.

The function takes the following parameters.

Syntax:

```
numpy.append(arr, values, axis)
```

```
graph TD; A["array([1, 2, 3, 4])"] --> C["append()"]; B["[5, 6, 7]"] --> C; C --> D["array([1, 2, 3, 4, 5, 6, 7])"]
```

Append Values to Array

- **arr** : Input array
- **values** : To be appended to arr. It must be of the same shape as of arr(excluding axis of appending)
- **axis** : The axis along which append operation is to be done. If not given, both parameters are flattened

```
import numpy as np
arr = np.array([[1,2,3],[4,5,6]])
print('First array:')
```

```
print(arr)

arr1 = np.array([7, 8, 9])
print('Second array:')
print(arr1)
print('Append elements to array:')
print(np.append(arr, arr1))

print('Append elements along axis 0:')
print(np.append(arr, [[7, 8, 9]], axis=0))

print('Append elements along axis 1:')
print(np.append(arr, [[5, 5, 5], [7, 8, 9]], axis=1))
```

output:

First array:

```
[[1 2 3]
 [4 5 6]]
```

Second array:

```
[7 8 9]
```

Append elements to array:

```
[1 2 3 4 5 6 7 8 9]
```

Append elements along axis 0:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Append elements along axis 1:

```
[[1 2 3 5 5 5]
 [4 5 6 7 8 9]]
```

7. numpy.insert

This function inserts values in the input array along the given axis and before the given index. If the type of values is converted to be inserted, it is different from the input array. Insertion is not done in place and the function returns a new array. Also, if the axis is not mentioned, the input array is flattened.

The insert() function takes the following parameters –

Syntax:

```
numpy.insert(arr, pos, values, axis)
```

- **arr** : Input array
- **pos** : The index before which insertion is to be made
- **values** : The array of values to be inserted
- **axis** : The axis along which to insert. If not given, the input array is flattened

```
import numpy as np
arr = np.array([[1,2],[3,4],[5,6]])
print('First array:')
print(arr)
```

```
print('Axis parameter not passed. The input array is flattened  
before insertion.')
```

```
print(np.insert(arr, 3, [11, 12]))
```

```
print('Axis parameter passed. The values array is broadcast  
to match input array.')
```

```
print('Broadcast along axis 0:')
```

```
print(np.insert(arr, 1, [11], axis=0))
```

```
print('Broadcast along axis 1:')
```

```
print(np.insert(arr, 1, 11, axis=1))
```

output:

First array:

```
[[1 2]
```

```
 [3 4]
```

```
 [5 6]]
```

*Axis parameter not passed. The input array is flattened before
insertion.*

```
[ 1  2  3 11 12  4  5  6]
```

*Axis parameter passed. The values array is broadcast to match
input array.*

Broadcast along axis 0:

```
[[ 1  2]
```

```
[11 11]
```

```
[ 3  4]
```

```
[ 5  6]]
```

Broadcast along axis 1:

```
[[ 1 11  2]
```

```
[ 3 11  4]
```

```
[ 5 11  6]]
```

8. `numpy.delete`

This function returns a new array with the specified subarray deleted from the input array. As in case of `insert()` function, if the `axis` parameter is not used, the input array is flattened. The function takes the following parameters –

Syntax:

```
numpy.delete(arr, obj, axis)
```

- `arr` : Input array
- `obj` : Can be a slice, an integer or array of integers, indicating the subarray to be deleted from the input array
- `axis` : The axis along which to delete the given subarray. If not given, `arr` is flattened

```
import numpy as np
```

```
a = np.arange(12).reshape(3,4)
```

```
print('First array:')
```

```
print(a)
```

```
print('Array flattened before delete operation as axis not  
used:')
```

```
print(np.delete(a, 5))
```

```
print('Column 1 deleted:')
```

```
print(np.delete(a, 1, axis=1))
```

output:

First array:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Array flattened before delete operation as axis not used:

```
[ 0  1  2  3  4  6  7  8  9 10 11]
```

Column 1 deleted:

```
[[ 0  2  3]
 [ 4  6  7]
 [ 8 10 11]]
```

9. numpy.unique

This function returns an array of unique elements in the input array.

The function can be able to return a tuple of array of unique values and an array of associated indices. Nature of the indices depend upon the type of return parameter in the function call.

Syntax:

```
numpy.unique(arr, return_index, return_inverse, return_counts)
```

- `arr` : The input array. Will be flattened if not 1-D array
- `return_index` : If True, returns the indices of elements in the input array
- `return_inverse` : If True, returns the indices of unique array, which can be used to reconstruct the input array
- `return_counts` : If True, returns the number of times the element in unique array appears in the original array

```
import numpy as np
a = np.array([5,2,6,2,7,5,6,8,2,9])
print('First array:')
print(a)
print('\n')

print('Unique values of first array:')
u = np.unique(a)
print(u)
print('\n')
```

```
print('Unique array and Indices array:')
u,indices = np.unique(a, return_index = True)
print(indices)
print('\n')

print('Return the count of repetitions of unique elements:')
u,indices = np.unique(a,return_counts = True)
print(u)
print(indices)
```

output:

First array:

[5 2 6 2 7 5 6 8 2 9]

Unique values of first array:

[2 5 6 7 8 9]

Unique array and Indices array:

[1 0 2 4 7 9]

Return the count of repetitions of unique elements:

[2 5 6 7 8 9]

[3 2 2 1 1 1]

String Functions

Standard string functions in Python's built-in library.

1. `numpy.char.add()`

This function performs elementwise string concatenation.

```
import numpy as np
print('Concatenate two strings:')
print(np.char.add(['hello'], [' xyz']))
print('\n')
print('Concatenation example:')
print(np.char.add(['hello', 'hi'], [' abc', ' xyz']))
```

output:

Concatenate two strings:

['hello xyz']

Concatenation example:

['hello abc' 'hi xyz']

2. `numpy.char.multiply()`

This function performs multiple concatenation.

```
import numpy as np
print(np.char.multiply('Hello', 3))
```

output:

Hello Hello Hello

3. `numpy.char.center()`

This function returns an array of the required width so that the input string is centered and padded on the left and right with fillchar.

```
import numpy as np
print(np.char.center('hello', 20, fillchar='*'))
```

output:

```
*****heLlo*****
```

4. `numpy.char.capitalize()`

This function returns the copy of the string with the first letter capitalized.

```
import numpy as np
print(np.char.capitalize('hello world'))
```

output:

```
Hello world
```

5. `numpy.char.title()`

This function returns a title cased version of the input string with the first letter of each word capitalized.

```
import numpy as np
print(np.char.title('hello how are you?'))
```

output:

```
Hello How Are You?
```

6. numpy.char.lower()

This function returns an array with elements converted to lowercase. It calls `str.lower` for each element.

```
import numpy as np
print(np.char.upper('hello'))
print(np.char.upper(['hello', 'world']))
```

output:

```
['hello' 'world']
```

```
Hello
```

7. numpy.char.split()

This function returns a list of words in the input string. By default, a whitespace is used as a separator. Otherwise the specified separator character is used to split the string.

```
import numpy as np
print(np.char.split('hello how are you?'))
print(np.char.split('India, is my ,country', sep=','))
```

output:

```
['hello', 'how', 'are', 'you?']
```

```
['India', ' is my ', 'country']
```

8. numpy.char.strip()

This function returns a copy of array with elements stripped of the specified characters leading and/or trailing in it.

```
import numpy as np
print(np.char.strip('ashok arora', 'a'))
```

```
print(np.char.strip(['arora', 'admin', 'java'], 'a'))
```

output:

shok aror

['ror' 'dmin' 'jav']

9. numpy.char.join()

This method returns a string in which the individual characters are joined by separator character specified.

```
import numpy as np
print(np.char.join(':', 'dmy'))
print(np.char.join([':', '-'], ['dmy', 'ymd']))
```

output:

d:m:y

['d:m:y' 'y-m-d']

10. numpy.char.replace()

This function returns a new copy of the input string in which all occurrences of the sequence of characters is replaced by another given sequence.

```
import numpy as np
print(np.char.replace('He is a good boy', 'is', 'was'))
```

output:

He was a good boy